

LumenTrace: A Mechanics-Free LED Slot Car Racing Game with Real-Time Physics and Immersive Audiovisual Feedback

Simon Völkl , Erik Dombrowsky, and Simon Frey

Abstract—Classic slot car systems offer an engaging tactile racing experience, yet they remain bound to physical vehicles on mechanical rails and treat light and sound as secondary embellishments. Simultaneously, minimalist LED racers reduce gameplay to button pressing and omit realistic physics, spatial audio and advanced track features. LumenTrace aims to unite the physical realism and playability of slot cars with the mechanical simplicity and immersive, state-driven audiovisual feedback that individually addressable LEDs make possible. The system is built on a distributed Raspberry Pi Pico and Raspberry Pi 4 architecture. A real-time physics engine models acceleration, friction, collision detection, and crash recovery from throttle and button inputs transmitted over a high-speed UART link. A priority-based LED rendering pipeline displays adaptive, state-dependent track and vehicle effects, complemented by a stereo audio mixer. The system meets its real-time thread budgets and low-latency requirements, resulting in positive user feedback during playtests, though it lacks cost-effectiveness for open-source developers. LumenTrace closes the gap between commercial slot car systems and minimalist LED racers, while providing highly performant, maintainable and safe system architecture. Future work focuses on cost reduction, improved playability, and game extensions.

Index Terms—Distributed Embedded Systems, LED Racing Game, Microcontroller, Real-Time Physics Engine, Slot Car Racing, Spatial Audio, WS2812B

I. INTRODUCTION

SLOT car racing has entertained enthusiasts for decades, but its classic form ties the experience to physical vehicles running on mechanical rails, where light and sound serve merely as peripheral embellishments. LumenTrace reimagines this concept as a mechanics-free racing simulator in which each vehicle is represented directly as a moving point of light on an individually addressable light-emitting diode (LED) strip. Built on a distributed architecture of a Raspberry Pi Pico and a Raspberry Pi 4 [1], the system couples a real-time physics engine with a priority-based LED rendering pipeline and a spatial audio mixer to deliver an immersive, state-driven racing experience without any moving mechanical parts. The complete source code and documentation are publicly available in the project's GitHub repository [2].

A. Motivation

Classic slot car systems deliver an engaging tactile racing experience, but they remain bound to physical vehicles running

The authors are with the Department of Electrical Engineering, Media, and Computer Science, Ostbayerische Technische Hochschule Amberg-Weiden, 92224 Amberg, Germany (e-mail: s.voelkl2@oth-aw.de; e.dombrowsky@oth-aw.de; s.frey@oth-aw.de). The project is part of the Physical Computing Course at OTH Amberg-Weiden and is supervised by Prof. Dipl.-Des. Martin Frey and Prof. Dr.-Ing. Ulrich Schäfer.

on mechanical rails. In these systems, light and sound are treated as peripheral add-ons rather than as core gameplay elements: engine sounds are typically monotone and decoupled from the vehicle's actual speed, and illuminated track sections merely blink in fixed patterns instead of responding to the race state [3]. Even highly digitalized products such as Carrera Digital 132 [4], Anki Overdrive [5] or Scalextric ARC PRO [6] retain physical cars and use audiovisual feedback only as a secondary layer.

LED-based racing games take the opposite approach and represent each vehicle directly as a moving light on an individually addressable strip [7], [8]. This removes all moving parts, yet existing projects stay deliberately minimalistic: gameplay is reduced to button mashing, and features such as a realistic physics model, spatially positioned audio, bidirectional lane changing, collision recovery, and modular track profiles are absent. As detailed in Section II, no existing system combines the physical realism and playability of slot cars with the mechanical simplicity and immersive, state-driven audiovisual feedback that addressable LEDs make possible. LumenTrace is motivated by closing precisely this gap.

B. Problem Statement

The central problem addressed by LumenTrace is the absence of a racing system that simultaneously provides the realistic physics and competitive playability of commercial slot cars and the mechanical simplicity and expressive, state-driven audiovisual feedback of addressable LED strips. Commercial slot car systems retain physical vehicles on mechanical rails and relegate light and sound to fixed, speed-decoupled embellishments, whereas existing LED racers reduce gameplay to acceleration pulses and omit a realistic physics model, spatial audio, bidirectional lane changing, collision recovery, and modular track profiles. LumenTrace therefore sets out to design and implement a mechanics-free LED slot car simulator that unifies these two design spaces on a low-latency, maintainable, and safe distributed embedded architecture.

C. Contributions

LumenTrace contributes a mechanics-free LED slot car simulator that unifies the strengths of both worlds on a distributed Raspberry Pi Pico and Raspberry Pi 4 architecture [1]. At its core is a real-time physics engine that models acceleration, friction, speed clamping, collision detection, and crash recovery, driven by analog throttle and digital button inputs transmitted over a high-speed Universal Asynchronous Receiver/Transmitter (UART) link [9]. On top of this, a priority-based LED rendering pipeline paints the track and



Fig. 1. LumenTrace: at the speed of light. Full-scale setup of the LED racing game showing the track, vehicles, controllers, and audiovisual system.

vehicles with layered, state-dependent effects, including adaptive vehicle colors, headlights, and brake lights that give the driver ambient feedback about speed [10], together with round counters realized through physically chained WS2812B matrices [11]. A vectorized, thread-isolated stereo audio mixer adds per-vehicle engine sounds whose pitch and stereo position follow the car's speed and track position, complemented by positional event sounds. Playability is extended beyond prior LED racers through a temporary bidirectional overtaking lane and modular track modules with individual driving profiles. Finally, the system is packaged for reproducible deployment via Docker [12] and ships with a hardwareless simulation interface for development and testing without physical hardware.

D. Document Outline

The remainder of this report is organized as follows. [Section II](#) surveys commercial slot car systems and open-source LED racing projects. [Section III](#) specifies the functional and non-functional requirements and derives the distributed system architecture. [Section IV](#) details the microcontroller setup, the UART communication interface, and the physical design. [Section V](#) describes the software architecture and the physics-based game mechanics, while [Section VI](#) covers the priority-based LED rendering pipeline and the real-time audio mixer. [Section VII](#) reports on the project management and task distribution, [Section VIII](#) evaluates the resulting system, and [Section IX](#) concludes and outlines future work.

II. RELATED WORK

LumenTrace sits at the intersection of two established domains: commercial slot car racing systems and open-source LED strip games. This section surveys both landscapes to contextualize the design decisions presented in the remainder of this report.

A. Existing Slot Car Systems

Commercial systems have steadily added electronics while keeping physical cars on mechanical rails. Carrera Hybrid [13] pushes toward a more digitalized and realistic racing simulation with collisions, overtaking, and modular track assembly. Carrera Digital 132 [4] offers button-triggered lane changes, lap counters, start lights, driver displays, and wireless controllers, but the vehicles remain fully physical rather than simulated. Scalextric ARC PRO [6] adds an app-based interface with pit stops, lap-time tracking, weather conditions, and multiple race modes, while reducing the controller to a brake and a lane-change button. Anki Overdrive [5] abandons fixed slots in favor of app-controlled autonomous cars on magnetically connectable, modular track segments.

Across these systems, light and sound serve as secondary embellishments rather than integral gameplay elements. Many hobbyist Carrera GO!!! [14] layouts add track-side lighting such as traffic signals, small LCD readouts, or vehicle head- and tail-lights, yet these effects are neither integrated into the race flow nor used to illuminate the lanes themselves. The Carrera GO!!! Light and Sound Looping Set [3] is representative: roughly 16 LEDs on each side of the looping blink red and white as a car passes, accompanied by a fixed engine sound effect. Both the blinking and the sound are static and remain decoupled from the actual vehicle speed. Even in the most digital products, audio is dominated by the mechanical friction noise of the miniature cars and is limited to short indicator cues at the start and on completing a lap.

B. LED-Based Projects and Racing Games

A parallel line of work explores addressable LED strips as minimalist, one-dimensional game displays driven by simple controllers. Line Wobbler [15] is an award-winning one-dimensional dungeon crawler whose custom controller consists

of a spring and a circa 5 m LED strip; it has been widely exhibited in museums and has inspired follow-up projects such as the Arduino-based crawler TWANG [16]. These titles demonstrate the appeal of novel input mechanics and physical space, yet they remain single-player experiences without competitive racing elements.

The most directly related project is the open-source Open LED Race (OLR) developed since 2019 [8]. OLR simulates a slot car track on a WS2812/WS2813 strip, representing each player as a colored light that advances by one acceleration pulse per button press, and supports up to four cars with optional slopes and gravity, a pit lane, and an electric-car mode requiring recharging [7], [17]. The project has evolved through numerous community forks: extended gravity, sound, and lighting effects [18], a layout with 3D-printed track elements focussing on design [19], an added 16×16 LED matrix display [20], a TinyGo reimplement [21], and wireless ESP-based remote controllers [22].

LumenTrace shares OLR's foundational ingredients of controller input, addressable LED strips, and microcontroller-based execution, but deliberately targets a different point in the design space. It omits OLR features such as gravity slopes, pit-lane recharging, and the electric-car mode, and instead introduces a realistic physics model, bidirectional lane changing, collision detection with crash recovery, priority-based immersive lighting, spatial audio, and modular track profiles. In doing so, LumenTrace bridges the realism and playability of commercial slot car systems with the flexibility and expressive audiovisual potential of LED-based games.

III. SYSTEM REQUIREMENTS AND ARCHITECTURE

This section defines the functional and non-functional requirements that guided the development of LumenTrace and derives the system architecture that satisfies them.

A. System Requirements

The system requirements define both functional capabilities and non-functional performance constraints that ensure LumenTrace meets user expectations and technical standards.

Functional Requirements: The initial project scope centered on developing a mechanics-free digital slot car racing system with a real-time control system, individually addressable red, green, and blue (RGB) LED strips for track and vehicle visualization, basic sounds, and utilizing the Raspberry Pi Pico 2 W microcontroller with PIO. Additionally, emphasis on design and playability was laid out to ensure an engaging user experience.

Non-Functional Requirements: The system was designed to meet the following quality and performance attributes:

- **Performance:** The system must render at greater than 24 Hz for human-perceivable smoothness. Controller inputs must be processed with minimal latency (less than 100 ms end-to-end) to provide responsive, immediate feedback during gameplay.
- **Scalability:** The modular software architecture must support straightforward extension with new track modules,

driving profiles, light and sound effects without requiring core system modifications.

- **Safety:** All electrical hazards must be mitigated through proper insulation and power management. Mechanical components must be designed to eliminate sharp edges and pinch points accessible to users.
- **Reliability and Availability:** The system must gracefully handle errors and automatically recover or restart. Cold boot initialization must complete within 120 s to minimize user wait time.
- **Maintainability:** The codebase must be well-documented with clear separation of concerns. Hardware and software components must be independently testable and replaceable without system-wide redesign. A simulation interface must be provided for development and testing without requiring physical hardware.
- **Usability:** The user interface must be intuitive and require minimal learning. Controller operation, track physics, and audiovisual feedback must be immediately comprehensible to new players. The game difficulty must be accessible to casual users while remaining engaging for experienced players.

B. High-Level Architecture

LumenTrace is built on a distributed two-tier architecture that splits the workload between an input-gathering microcontroller and a central core host [1], as depicted in Figure 2. The input controller captures the physical user commands and streams them over a serial connection to the main processing unit. The core host then executes the central application to resolve the game state and drive the output devices.

This separation of concerns offloads the latency-critical, high-frequency input sampling from the core host, allowing it to dedicate its computational resources to processing game state and real-time feedback without incurring polling overhead. Structuring the system into independent, well-defined subsystems also keeps it modular, maintainable, and scalable.

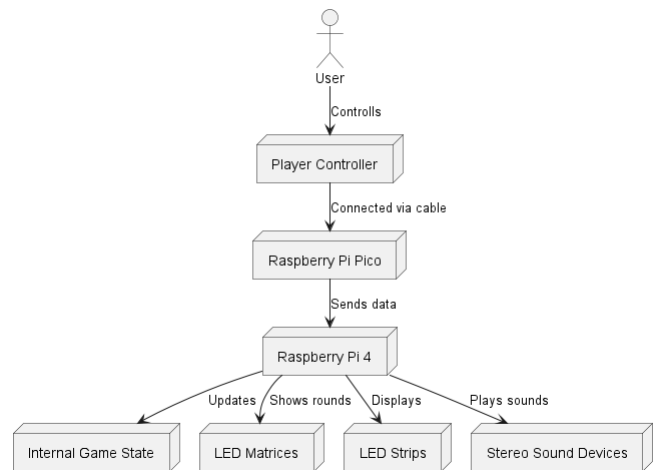


Fig. 2. High-level system architecture of LumenTrace, showing the distributed setup with the Raspberry Pi Pico as input microcontroller and the Raspberry Pi 4 as core host.

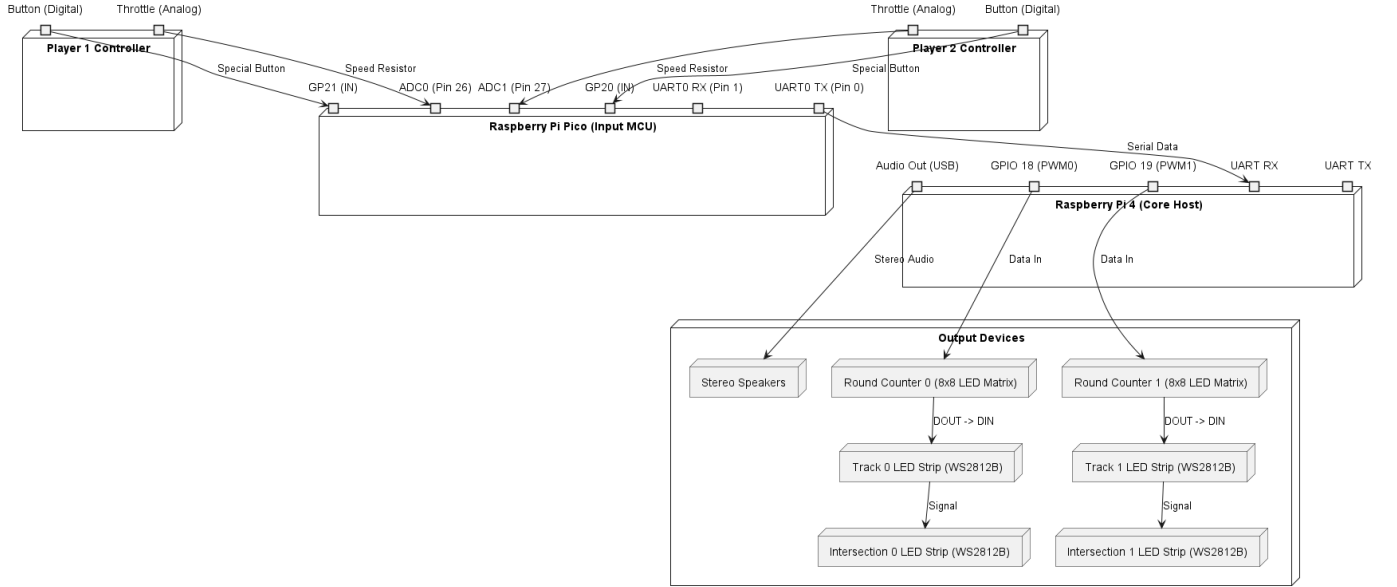


Fig. 3. Hardware setup of LumenTrace: The Raspberry Pi Pico handles controller inputs and transmits them to the Raspberry Pi 4, which drives the output devices.

IV. HARDWARE IMPLEMENTATION

This section describes the hardware realization of LumenTrace, covering the distinct roles of the two microcontrollers, the serial communication interface between them, and the physical construction of the track and controllers.

A. Microcontroller Setup

As illustrated in Figure 3, the specific hardware configuration establishes the low-level execution environments for both devices. The Raspberry Pi Pico acts as the dedicated input microcontroller running MicroPython firmware [23]. It permanently and latency-free samples the physical controllers, utilizing its internal 12-bit analog-to-digital converters (ADCs) channels to read the analog throttles and its General Purpose Input/Output (GPIO) pins to detect digital button presses.

Conversely, the Raspberry Pi 4 functions as the core host, booting its operating system and the main Python application from a micro-SD card to enable fully autonomous physical startup. The Pi 4 processes the inputs transmitted by the Pico to maintain the internal game state and execute physics ticks. It translates this game state into audiovisual feedback by actuating the LED strips and matrices [11], and by driving the stereo audio mixer.

B. Communication Interface

The Pico and the Pi 4 communicate over a high-speed UART link [9], [24]. On every polling cycle, the Pico serializes the current state of all connected controllers into a compact JavaScript Object Notation (JSON) structure [25] and transmits it to the Pi 4. Delegating input acquisition and serialization to the Pico relieves the core host from polling overhead, so that it can consume the most recent controller state directly instead of sampling the hardware itself. As

shown in Figure 4, each controller entry reports a unique `controller_id`, the raw analog throttle value `adc_0`, and the digital button state `dig_0`.

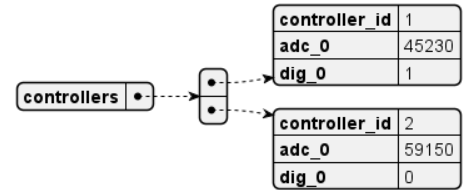


Fig. 4. Example of the JSON structure transmitted from the Pico to the Pi 4 over UART, containing the current state of two controllers.

At an update rate of 50 Hz, the protocol tolerates sporadic transmission losses: roughly every twentieth packet is dropped or unreadable, yet a single missing update is uncritical because the game engine simply continues with the last known controller state until the next valid packet arrives.

C. Physical Design and Controllers

The physical setup is built on a black wooden base plate of 150 cm × 80 cm, spray-painted in blue, purple, and white to create a retro-futuristic backdrop. The track layout reuses Carrera GO!!! [14] track modules, along with two 5 m highly diffusing LED strips carrying 50 LEDs per meter (250 LEDs in total), to visualize the lanes, vehicles and intersections. All components were rigidly fastened using screws, cyanoacrylate adhesive, and double-sided tape to preserve transportability, and the looping module was additionally reinforced with fishing line to prevent fractures.

The original Carrera GO!!! hand controllers were modified from their built-in variable resistors into signal sources for the Pico. Each throttle is wired as a voltage divider feeding one

of the Pico's ADC channels, yielding the raw analog value transmitted as `adc_0`, while an added push button provides the digital lane-change signal (`dig_0`).

Custom 3D-printed enclosures, styled to match the sci-fi aesthetic, conceal the Raspberry Pi 4, the Pico, the internal wiring, and house the stereo speakers. Beyond their visual role, these enclosures also serve a safety function: together with the only 3 A power draw per LED strip, they cover the electronics to prevent user contact and eliminate sharp edges and pinch points.

V. SOFTWARE IMPLEMENTATION AND GAME MECHANICS

This section describes the modular software architecture of the core application and the physics-based game mechanics it implements.

A. Software Architecture

The core application on the Raspberry Pi 4 follows an object-oriented design and is decomposed into loosely coupled Python packages, each with a well-defined responsibility, as illustrated in Figure 5. The `Game` package holds the core game logic, comprising vehicle physics, track management, and players. The `Controller` package translates player controller input into game state changes, while the `Display` package manages the LED effects and visual feedback. The `Sound` package provides the real-time, threaded audio mixer, and the `Logger` package centralizes debugging output to log files and MQTT [26], [27]. Finally, the `Simulation` package enables testing and visualization of the game mechanics without physical hardware. This separation of concerns keeps the system modular, maintainable, and straightforward for further extensions.

Python was chosen as the implementation language to maximize development velocity, readability, and access to mature hardware, audio, and testing libraries within the given project timeframe, accepting the Global Interpreter Lock (GIL) as a known trade-off against lower-level alternatives such as C/C++.

Because real-time audio and high-frequency physics impose strict timing guarantees, the application is partitioned into independent threads to mitigate contention on the GIL [28]; where thread-level parallelism proved insufficient, the audio path is further decoupled behind an asynchronous command queue rather than migrated to multiprocessing, keeping the shared game state simple and lock-free across subsystems. The physics loop runs at 50 Hz, the LED display pipeline at 33 Hz, and controller input polling at 50 Hz.

The latency-critical physics thread is strictly separated from the peripheral driver calls of the controller and display subsystems, and the audio interface is further isolated behind an asynchronous command queue [29], as detailed in Section VI-B.

B. Controller Inputs and Speed Calculation

Each 20 ms, the raw analog throttle value `forwardPress` reported by the Pico is mapped to a vehicle acceleration through linear interpolation [30], scaling the input range

[42000, 65536] onto the output range [0, 100] of the acceleration a :

$$a(\text{forwardPress}) = \frac{\text{forwardPress} - 42000}{65536 - 42000} \cdot 100. \quad (1)$$

The speed v is then integrated per tick as

$$v += a \cdot a_{\text{mult}} \quad (2)$$

and clamped to the range $[-v_{\text{max}}, +v_{\text{max}}]$ to prevent unstable values at high update rates, where a_{mult} is a configurable acceleration multiplier, and v_{max} is the maximum speed limit for both driving directions. To emulate realistic driving behavior, an adjustable friction coefficient [31] continuously dampens the current speed

$$v *= (1 - \text{friction}). \quad (3)$$

C. Physics

LumenTrace models several physics-based mechanics that emulate the behavior of classic slot cars [32]. The vehicle position is derived from speed using the active game-tick interval. When the position exceeds the length of the full track lane, the *lap counter* is incremented, and the position wraps around to zero.

Lane changing is realized through a temporary bidirectional center lane introduced by intersection modules. A change is only permitted when the driving profile of the current line allows it, but never within the final `laneChangeWindow` units before a module ends. A vehicle first switches from an outer lane to the middle lane and, after travelling at least one `laneChangeWindow`, may manually merge onto the target outer lane; otherwise the vehicle merges automatically before the module ends. During a switch, relative progress within the module is preserved by proportional conversion, where the target position equals the source progress ratio scaled by the target line length, keeping vehicles visually and physically aligned across lanes of differing lengths or radii.

Falling is triggered immediately when a vehicle's speed or acceleration violates the driving profile of the active line, whose constraints are subdivided across curve and looping segments so that difficulty scales smoothly rather than abruptly at module entry. *Collision detection* is evaluated per lane using a distance metric: if two vehicles come within a configured crash distance, the leading vehicle loses control and falls while the trailing vehicle continues unharmed, simulating a rear-quarter impact that preserves playability.

After a crash, a multi-stage *respawn* procedure grants a collision-free placement with enough track to rebuild momentum for a possible looping in following modules. Fallen vehicles become inactive for a fixed number of respawn ticks and are ignored by the race. On respawn from a position inside a looping, a step back by one module is performed to avoid an immediate consecutive crash; the vehicle then attempts position zero on its preferred lane, falling back to other unoccupied lanes and, if necessary, to the track start. If no lane is available, the vehicle stays inactive and retries on the next tick until a position is free. Speed, acceleration, and the lane-change state are reset.

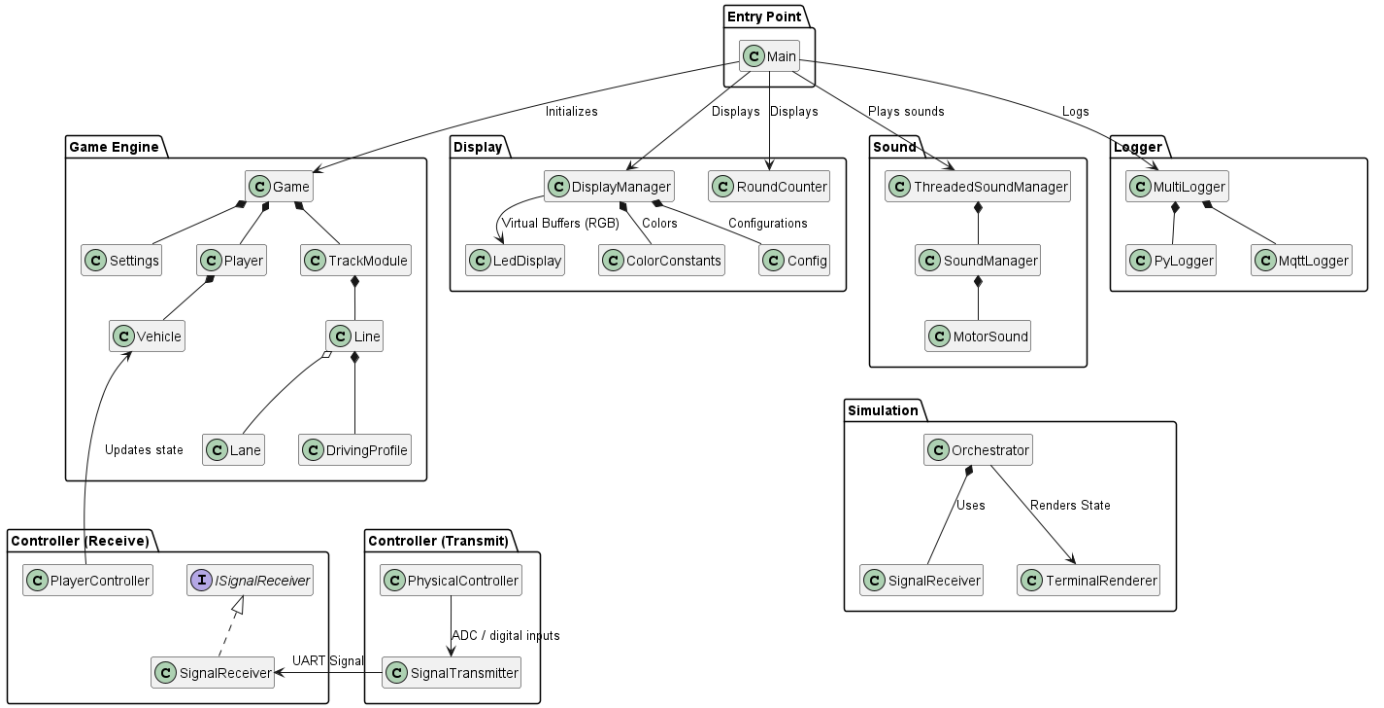


Fig. 5. Object-oriented software architecture of the LumenTrace core application, showing the modular package design.

D. Deployment

For reproducible and hardware-independent operation, the LumenTrace application is packaged as a Docker container [12] on the Raspberry Pi 4. Running in privileged mode, the container bundles all system-level dependencies and grants the direct hardware access required for GPIO, LED, UART, and audio peripherals [33], ensuring consistent behavior across deployments. Privileged mode is a deliberate security trade-off: it effectively disables container isolation and grants the container full access to host devices, which is acceptable for this self-contained, single-purpose appliance on a dedicated, non-networked device, but would need to be replaced by fine-grained device and capability mappings (e.g., explicit `--device` and `--cap-add` rules) in any multi-tenant or network-exposed deployment. A systemd unit binds the container to the host lifecycle, starting it automatically on boot and restarting it on failure, which satisfies the cold-boot and reliability requirements from Section III-A.

E. Simulation and Testing

To validate the game logic without physical hardware, LumenTrace ships a text-based simulation runner that renders the full game state live in the terminal, including player and vehicle telemetry and an ASCII lane visualization. It is complemented by a `pytest`-based unit test suite [34] covering the core game rules, controller inputs and output devices. Together, they satisfy the maintainability requirement of independent, hardwareless testability from Section III-A.

VI. MULTIMEDIA SUBSYSTEMS: LED AND AUDIO CONTROL

This section covers the two multimedia subsystems of LumenTrace: the priority-based LED rendering pipeline and the real-time stereo audio mixer, together with the hardware constraints that shaped their design.

A. LED Rendering

Driving the two round-counter matrices together with the two track strips initially required four independent `PixelStrip` objects of the `rpi_ws281x` library on separate control lines (GPIO 10/SPI, GPIO 21/PCM, and GPIO 18/19/PWM). This approach failed because the underlying C driver [11] cannot mix PWM, SPI, and PCM peripherals concurrently within a single process, is hardware-capped at two active channels, and produced DMA register collisions and memory-mapping failures when initializing multiple strips on the same DMA channel.

The bottleneck was resolved by physically daisy-chaining all LED components onto the two PWM channels only. Along each of the two channels, the signal from the Pi 4's GPIO first drives the round-counter matrix, then the main lane strip, and finally the additional intersection strip, with the connecting wires hidden beneath the track, as depicted in Figure 3.

A display manager translates the game state into visual output on the track LED strips by composing several layers from low to high priority, so that higher-priority effects override lower ones. The hierarchy is applied in the following order:

- 1) *Track module base*: faint purple main lanes.
- 2) *Intersection module*: lanes on intersection modules with a light pink overlay.

- 3) *Intersection animation*: stronger pink overlay for indicating a possible lane change.
- 4) *Start line*: starting position with a white overlay.
- 5) *Round advance*: blinking starting position on lap completion.
- 6) *Inactive vehicles*: respawning vehicles displayed as dimmed gray.
- 7) *Active vehicle*: custom vehicle color with a speed-dependent color interpolation as speed warning [10].
- 8) *Front lights*: white headlights, disabled by default.
- 9) *Rear lights*: red brake lights when no throttle is applied.

To decouple this logic from the constrained hardware, the display layer builds on a virtual led abstraction that maps predefined segments of a physical strip onto logical lanes. This lets the display manager address the lap counter, main lane, and overtaking lane independently while hiding the physical daisy-chain and the two-channel limitation from the game logic. At each tick, the display manager writes RGB tuples into per-lane virtual buffers, and renders them onto the physical `PixelStrip` objects of the `rpi_ws281x` library.

B. Real-Time Audio Mixer

Audio in LumenTrace is mixed live and played back in stereo so that the soundscape reflects the race state at every moment [35], [36]. A low-level sound manager combines sounds through a single output stream, controlling per-sound volume, dynamic pitch, and stereo panning independently [37], [38], [39]. Each vehicle owns a continuous engine sound whose pitch and loudness follow its speed, gains an additional layer while accelerating, and pans between the speakers according to the current track module's defined stereo ratio, complemented by positional one-shot effects, e.g., for the start sequence, laps, lane changes, crashes, and warnings [40].

Problem: initially, routing audio through the Pi's AUX jack produced significant background noise, which was resolved by switching to a USB audio interface. Moreover, executing the Python audio callbacks synchronously alongside high-frequency game ticks, rendering, and hardware polling caused GIL starvation and lock contention that manifested as severe sound stuttering after just a few seconds running the game. *Solution*: the audio interface is isolated behind a thread-safe asynchronous command queue [29], [28]. A non-blocking sound manager proxy enqueues sound commands as fire-and-forget entries without ever blocking the caller, and a dedicated worker thread applies them to the underlying mixer. Together with minimal 1 ms sleep intervals in the display loop that prevent audio-thread starvation, this stabilized the output under concurrent workloads.

VII. PROJECT MANAGEMENT

The LumenTrace project was executed by a three-person team, with each member contributing to different aspects of the project. The lead developers for each component are indicated in Table I, while a reviewer was assigned to each task to ensure hardware and software quality.

The development of LumenTrace followed an incremental path, prioritizing a basic architecture and core functionality before implementing additional features and working on

enhancements. The project was structured into four distinct milestones to manage dependencies and verify integration at each phase.

- *Milestone 1: Proof of Concept (POC)*: Establishment of the physical Carrera GO!!! [14] track, development of Raspberry Pi Pico firmware using MicroPython, implementation of analog controller input reading, establishment of UART serial communication, and design of the JSON-based data serialization and parsing.
- *Milestone 2: Base Game Implementation*: Software architecture with modular packages, core physics engine with acceleration, friction, and speed clamping, vehicle lane management and position tracking, collision detection via distance metrics, multithreaded game loop architecture, LED strip control, display rendering pipeline with priority-based layer system, modular track design with individual driving profiles, settings system, unit testing framework, and centralized logging via file and MQTT interfaces.
- *Milestone 3: Enhancements and Feature Expansion*: Advanced physics for lane changes, respawn system, intersection modules, virtual LED strip abstraction for hardware constraint resolution, round counters, vehicle lighting effects including headlights, rear brake lights, and identification lights, real-time vectorized stereo audio mixer with volume and stereo panning controls, threaded sound manager, per-vehicle engine sounds with speed-responsive pitch, event-driven sound effects, Docker containerization, 3D-printed controller enclosures and component styling, and performance optimization to prevent thread starvation.
- *Milestone 4: Documentation and Finalization*: Comprehensive technical report with architecture diagrams and system analysis, usage guides, technical report documentation, and project preparation for submission.

The division of responsibilities between the three team members is outlined in Table I. In general, Simon Völkl focused on the software implementation, Erik Dombrowsky concentrated on the hardware wiring and physical setup, while Simon Frey contributed mainly to the 3D printing and styling of the components.

VIII. EVALUATION

This section evaluates LumenTrace against its non-functional requirements from Section III-A, assessing real-time performance, user experience, cost, and remaining limitations.

A. Performance Analysis

The LumenTrace system achieves real-time performance across all critical subsystems, meeting or exceeding the specified non-functional requirements.

The system operates three independent processing threads at target frequencies of 50 Hz for the game engine physics, 33 Hz for the LED display pipeline, and 50 Hz for controller input polling. These frequencies are configured setpoints rather than instrumented measurements; they were selected to exceed

TABLE I
TASK DISTRIBUTION MATRIX

Milestone	Task	S. Völkl	E. Dombrowsky	S. Frey
M1: POC	Physical Track Setup	Lead	Lead	-
	Controller Modification	Review	Lead	-
	Pico Setup	Review	Lead	-
	JSON Schema Serialization	Lead	Lead	-
	JSON Schema Parsing	Lead	Review	-
	Microcontroller Wiring with UART	Review	Lead	-
M2: Base Game	Software Architecture	Lead	Review	-
	Logging	Lead	Review	-
	Unit Testing	Lead	-	-
	Console Simulation	Lead	-	-
	Physics Engine	Lead	Review	-
	Track Modules with Driving Profiles	Lead	Lead	Lead
	Settings System	Lead	Review	-
	Vehicle Lane Management	Lead	Review	Review
	Collision Detection	Lead	Review	-
	Multithreading	Lead	Review	-
	LED Strip Control	Lead	Lead	Lead
M3: Enhancements	Advanced Physics	Lead	Review	Review
	Respawn System	Lead	Review	Review
	Physical Intersection Modules	Review	Lead	-
	Virtual LED Strips	Lead	Review	-
	Round Counter Wiring	Review	Lead	Lead
	Round Counter Logic	Lead	Review	-
	Vehicle Lighting Effects	Lead	Review	Review
	Sound Samples	Lead	Lead	Lead
	Sound Engineering	Lead	Lead	-
	Docker Containerization	Lead	-	-
M4: Documentation	3D Printing/Styling	Review	Lead	Lead
	Technical Report	Lead	Review	Review
	Presentation Preparation	Lead	Lead	Review
	Usage Guides	Lead	-	-

the human perception threshold of circa 24 Hz [41], [42] and produced smooth, flicker-free visual output perceived as continuous motion during operation. A rigorous quantitative characterization of the frequencies actually sustained under load, including their timing jitter, remains future work.

Controller inputs demonstrate minimal end-to-end latency from physical button press through UART transmission, game logic processing, and LED display update. The system responds in 70 ms to 100 ms to throttle and lane-change commands with no perceptible delay; these figures were obtained by counting frames during a live run rather than through instrumented profiling. This responsiveness is maintained even during occasional UART transmission failures, which were estimated at approximately 5 % of packets by counting dropped or unreadable packets over a live run, as the system prioritizes the most recent controller state rather than queuing stale data. A precise, instrumented measurement of the end-to-end latency and packet-loss rate is planned for future evaluation.

The optional file and MQTT logging subsystems introduce measurable performance overhead due to I/O blocking and possible network latencies. Logging should remain disabled in production environments, else performance degradation and occasional 50 ms–200 ms lag spikes during log message transmission can occur; these spikes were observed by systematically measuring execution times during live runs. Deferred asynchronous logging could mitigate this overhead in future iterations.

B. User Experience and Feedback

The feedback reported here is anecdotal rather than the result of a controlled user study. It was gathered informally from the three developers and a small, unstructured group of casual observers who played the game during development, without a defined task protocol, participant sampling, or standardized questionnaire. Under these conditions, the combination of immersive LED lighting effects, spatially-positioned audio, and responsive physics was consistently perceived as compelling and engaging. The retro-futuristic aesthetic, with its purple and blue color scheme and sci-fi-styled 3D-printed components, resonated well with both casual observers and competitive players. Iterative refinements to speed limit transitions and audio volume levels were reported to improve overall player satisfaction and reduce jarring gameplay moments. A formal usability evaluation with a defined participant sample, structured tasks, and validated questionnaires (e.g., the System Usability Scale) is required to substantiate these observations and to operationalize the usability requirement from [Section III-A](#); conducting it is left to future work. Although the system currently relies on Carrera GO!!! [14] track modules, custom modules are planned for future development to enhance flexibility and independence from proprietary track systems. For museum exhibitions or trade show presentations, further concealment of cables is recommended to ensure safety and prevent potential malfunctions like loose connections.

C. Cost Analysis

Table II summarizes the approximate hardware cost of the LumenTrace prototype per component. A total cost of **310 €** was calculated, indicating a relatively high-cost prototype. Further cost reductions could be achieved by switching to lower-cost LED strips, creating custom track modules, or sourcing components in bulk.

TABLE II
HARDWARE COST OVERVIEW

Component	Unit Cost	Quantity	Total Cost
Raspberry Pi 4 (Core Host)	40 €	1	40 €
Raspberry Pi Pico (Input MCU)	5 €	1	5 €
32GB Micro-SD card	16 €	1	16 €
USB sound card	8 €	1	8 €
WS2812B diffusing LED strip	57 €	2	114 €
WS2812B 8x8 LED matrix	4 €	2	8 €
Stereo speakers (and subwoofer)	25 €	1	25 €
Carrera GO!!! track module	1 €	20	20 €
Carrera GO!!! controller	10 €	2	20 €
Wooden base plate	10 €	1	10 €
3D-printing PLA filament	6 €	1	6 €
Wiring and mounting equipment	2 €	3	6 €
Paint (spray and brush)	8 €	4	32 €
Total			310 €

D. Challenges and Limitations

Beyond the display, audio, and logging bottlenecks addressed earlier, a few conceptual limitations remain. The vertical height of the diffusing LED channel ruled out physically intersecting overtaking lanes, which is why overtaking relies on a temporary bidirectional center lane rather than true crossing paths. The abrupt transitions between the speed limits of adjacent track modules can still provoke unavoidable crashes when a vehicle enters a stricter profile at excessive speed, a shortcoming, which a smoothed limit interpolation could mitigate. The system is further bound to the two physical LED channels imposed by the `rpi_ws281x` driver [11], which the daisy-chained virtual LED design accommodates but does not eliminate, capping the number of independently wired strips. These constraints do not affect core gameplay but define the current envelope of the prototype. Enabling others to participate on the project, as in the OLR project [17], would require a more cost-effective and modular system design.

IX. CONCLUSION AND FUTURE WORK

LumenTrace demonstrates that the tactile appeal of classic slot car racing can be reproduced without moving parts by representing each vehicle as a light on an addressable LED strip. On a distributed Raspberry Pi Pico and Raspberry Pi 4 architecture [1], the system unifies a real-time physics engine featuring acceleration, friction, collision detection, and crash recovery, a priority-based LED rendering pipeline with adaptive, state-driven effects [10], and a vectorized, thread-isolated stereo audio mixer that positions engine and event sounds in space. Packaged for reproducible deployment via Docker [12] and validated through a hardwareless simulation interface and unit tests, the prototype meets its performance

targets and delivers a smooth, immersive, and low-latency racing experience that closes the gap between commercial slot car systems and minimalist LED racers.

Besides cost reduction and playability enhancements, future work on the system should include a rigorous empirical evaluation, comprising instrumented measurements of the sustained thread frequencies, end-to-end latency, and packet-loss rate under load, as well as a formal usability study with a defined participant sample and standardized questionnaires. Beyond evaluation, additional game features are envisioned. Physical controllers with vibration modules would add haptic feedback, while the collected MQTT telemetry could be visualized on a companion web or mobile dashboard [43], [44]. Gameplay could be enriched with special abilities beyond the overtaking lane, such as nitro boosts or projectiles, collectible coins, and additional game modes with environmental effects like storms or rain [6]. On the technical side, AI opponents trained with reinforcement learning, smoothed speed-limit transitions to prevent abrupt collisions, further ambient enhancements through additional 3D-printed components, and custom, magnetically connectable track modules [5] would extend both playability and independence from proprietary track systems.

REFERENCES

- [1] Makers of The MagPi magazine, *The Official Raspberry Pi Handbook 2025: Astounding projects with Raspberry Pi computers*. Raspberry Pi Press, 2024, electronic book. [Online]. Available: <https://search.ebscohost.com/login.aspx?direct=true&scope=site&db=nlebk&AN=3858359>
- [2] S. Völkl, E. Dombrowsky, and S. Frey. (2026) Lumentrace github repository. [Online]. Available: <https://github.com/s-voelkl/LumenTrace>. GitHub Repository.
- [3] C. T. GmbH. (2026) Carrera go light and sound looping set. [Online]. Available: <https://carrera-toys.com/en/products/20061661-light-and-sound-looping-set>. Official Website.
- [4] —. (2026) Carrera digital 132. [Online]. Available: <https://carrera-toys.com/en/collections/landing-page-digital-132-ar-pimcore-import>. Official Website.
- [5] I. Anki. (2026) Anki overdrive. [Online]. Available: <https://anki.bot/pages/meet-overdrive>. Official Website.
- [6] H. H. Limited. (2026) Scalextric digital - arc pro platinum gt race set. [Online]. Available: <https://uk.scalextric.com/products/scalextric-arc-pro-platinum-gt-set-c1413m>. Official Website.
- [7] GBarbarov. (2023) Open led race original 2 player version 2019. [Online]. Available: <https://projecthub.arduino.cc/gbarbarov/open-led-race-bb534b>. Arduino Project Hub.
- [8] M. F. Pereira and M. S. Oyamada, “Applying arduino and individually addressable led strips to the development of interactive materials to promote computer science undergraduate course,” in *Congresso Latino-Americano de Software Livre e Tecnologias Abertas (Latinoware)*. SBC, 2023, pp. 97–101.
- [9] E. Peña and M. G. Legaspi, “Uart: A hardware communication protocol understanding universal asynchronous receiver/transmitter,” *Visit Analog*, vol. 54, no. 4, pp. 1–5, 2020.
- [10] A. Meschtscherjakov, C. Döttlinger, C. Rödel, and M. Tscheligi, “Chaselight: ambient led stripes to control driving speed,” in *Proceedings of the 7th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*, ser. AutomotiveUI ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 212–219. [Online]. Available: <https://doi.org/10.1145/2799250.2799279>
- [11] J. Garff, *rpi_ws281x Documentation*, [Online]. Available: https://github.com/jgarff/rpi_ws281x, Online, 2023.
- [12] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux J.*, vol. 2014, no. 239, march 2014.
- [13] C. T. GmbH. (2026) Carrera hybrid racetrack. [Online]. Available: <https://carrera-toys.com/en/pages/overview-page-hybrid-overview>. Official Website.

- [14] —. (2026) Carrera toys official website. [Online]. Available: <https://carrera-toys.com/>. Official Website.
- [15] R. Baumgarten. (2023) Line wobbler. [Online]. Available: <https://www.wobblylabs.com/line-wobbler>. Official Website.
- [16] Critters. (2020) Twang github repository. [Online]. Available: <https://github.com/Critters/TWANG>. GitHub Repository.
- [17] O. L. R. Project. (2025) Open led race project. [Online]. Available: <https://openledrace.net/>. Official Website.
- [18] ArminJo. (2025) Openledrace extended version. [Online]. Available: <https://github.com/ArminJo/OpenledRace>. GitHub Repository.
- [19] H. F. e.V. (2024) Hackerspace open led race at night of science 2024. [Online]. Available: https://www.hackerspace-ffm.de/wiki/images/LED_Race.pdf. Article.
- [20] F. Mocq. (2024) Un afficheur pour la course de leds open led race. [Online]. Available: <https://www.framboise314.fr/un-afficheur-pour-la-course-de-leds-open-led-race/>. Article.
- [21] D. Esteban. (2019) Tinygo implementation of open led race. [Online]. Available: <https://github.com/conejoninja/ledrace>. GitHub Repository.
- [22] G. Martin. (2020) Open-led-race wireless remote. [Online]. Available: <https://github.com/gmag11/OpenLedRace-Wireless-Remote>. GitHub Repository.
- [23] D. P. George, M. authors, and contributors, *MicroPython Documentation*, [Online]. Available: <https://docs.micropython.org/en/>, MicroPython, 2026.
- [24] C. Liechti, *PySerial Documentation*, [Online]. Available: <https://pyserial.readthedocs.io/en/>, Online, 2020.
- [25] JSON.org. (2026) Introducing json. [Online]. Available: <https://www.json.org/json-en.html>.
- [26] MQTT.org. (2026) Mqtt: The standard for iot messaging. [Online]. Available: <https://mqtt.org/>.
- [27] R. Light and E. Foundation, *paho-mqtt Documentation*, [Online]. Available: <https://pypi.org/project/paho-mqtt/>, Eclipse Foundation, Online, 2026.
- [28] R. Eggen and M. Eggen, "Thread and process efficiency in python," in *Proceedings of the international conference on parallel and distributed processing techniques and applications (PDPTA)*. The Steering Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp), 2019, pp. 32–36.
- [29] P. Tsigas and Y. Zhang, "A simple, fast and scalable non-blocking concurrent fifo queue for shared memory multiprocessor systems," in *Proceedings of the thirteenth annual ACM symposium on Parallel algorithms and architectures*, 2001, pp. 134–143.
- [30] A. Zheng and A. Casari, *Feature engineering for machine learning: principles and techniques for data scientists*. "O'Reilly Media, Inc.", 2018.
- [31] M. Kane and V. Edmondson, "Tire/road friction prediction: Introduction a simplified numerical tool based on contact modelling," *Vehicle system dynamics*, vol. 60, no. 3, pp. 770–789, 2022.
- [32] S. L. Skaar and H. Vågsether, "A slot car framework for software engineering education," in *Robotics in Education*, R. Balogh, D. Obdržálek, and N. Fachantidis, Eds. Cham: Springer Nature Switzerland, 2025, pp. 380–391.
- [33] B. Croston, *Raspberry Pi GPIO Python Documentation*, [Online]. Available: <https://pypi.org/project/RPi.GPIO/>, Online, 2022.
- [34] H. Krekel and pytest-dev team, *pytest Documentation*, [Online]. Available: <https://docs.pytest.org/en/>, pytest-dev, Online, 2026.
- [35] D. Gärdenfors, "Designing sound-based computer games," *Digital Creativity*, vol. 14, no. 2, pp. 111–114, 2003. [Online]. Available: <https://doi.org/10.1076/digc.14.2.111.27863>
- [36] C. Gan, H. Zhao, P. Chen, D. Cox, and A. Torralba, "Self-supervised moving vehicle tracking with stereo sound," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.
- [37] spatialaudio and M. Geier, *sounddevice Documentation*, [Online]. Available: <https://python-sounddevice.readthedocs.io/en/>, Online, 2024.
- [38] B. Bechtold and M. Geier, *soundfile Documentation*, [Online]. Available: <https://python-soundfile.readthedocs.io/en/>, Online, 2026.
- [39] C. Harris, K. Millman, S. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. [Online]. Available: <https://doi.org/10.1038/s41586-020-2649-2>
- [40] Pixabay. (2026) Pixabay: Stunning royalty-free images and royalty-free stock. [Online]. Available: <https://pixabay.com/>.
- [41] J. Y. C. Chen and J. E. Thropp, "Review of low frame rate effects on human performance," *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, vol. 37, no. 6, pp. 1063–1076, 2007.
- [42] F. Pazhoohi and A. Kingstone, "The effect of movie frame rate on viewer preference: An eye tracking study," *Augmented Human Research*, vol. 6, no. 1, p. 2, 2021.
- [43] S. Völkl, J. Schieder, S. Rosner, A. T. Vu, and C. P. Neumann. (2026) Sensiq github repository. [Online]. Available: <https://github.com/s-voelkl/Sensiq>. [GitHub Repository].
- [44] S. Völkl and J. Schieder. (2026) Bitebound github repository. [Online]. Available: <https://github.com/s-voelkl/BiteBound>. [GitHub Repository].